

机器学习笔记

Leonhard Hsiao

2025 年 12 月 20 日

目录

1 特征选择和特征降维	1
1.1 特征选择	1
1.1.1 特征子集生成的搜索策略	2
1.1.2 特征评价准则	3
1.1.3 特征选择方法	6
1.2 特征提取	9
1.2.1 特征提取概述	9
1.2.2 主成分分析 (PCA)	10
1.2.3 SVD 分解	12
1.2.4 LLE (Locally Linear Embedding)	12
1.2.5 NMF (Non-negative Matrix Factorization)	14
1.2.6 KPCA	15

1 特征选择和特征降维

在机器学习任务中，原始数据往往拥有极高的维度，进行降维处理主要基于以下四个原因：

- **维度灾难**：随着维度 d 的增加，样本空间体积呈指数级膨胀，导致数据在高维空间变得极其稀疏，模型就很难学到规律，而且有可能把噪声当作数据学习。
- **冗余与噪声**：噪声是指某些特征对于预测任务是无用的，还有可能干扰学习过程；某些特征之间存在高度相似性，比如年龄和出生日期，同时考虑他们两个是没有必要的。
- **计算效率**：减少特征数量可以显著降低模型训练和推理时的计算复杂度，节省内存与存储空间。

虽然两者都旨在降低维度，但在处理手段和结果上有本质区别。特征选择是从原始特征集合中选取一个子集，保持特征的原有意义；特征提取类似于线性映射 $\mathbb{R}^n \rightarrow \mathbb{R}^m$ ，把数据从高维的特征空间映射到低维空间，过程会创造新的特征。

1.1 特征选择

定义 1.1 (特征选择问题). 特征选择 (Feature Selection) 是指从原始的 D 维特征集合中，挑选出一个包含 d 个特征 ($d < D$) 的子集来表达模式的过程。其核心目标是在尽量减少信息损失的前提下：

- 提高数据的压缩率；
- 提升学习机的预测性能（泛化能力）；
- 加快学习速度并降低计算开销。

特征选择通常是一个迭代循环的过程，包含四个核心环节：

1. **子集生成**：按照某种搜索策略产生候选的特征子集。
2. **子集评价**：利用评价准则 J 对候选子集进行打分，衡量其优劣。
3. **停止准则**：判断当前搜索是否达到预期目标（如达到指定特征数或评价指标不再提升）。
4. **结果验证**：在独立的测试集上验证最终选定子集的有效性。

1.1.1 特征子集生成的搜索策略

从 D 维特征中选择 d 维特征，其可能的子集个数为组合数：

$$C_D^d = \frac{D!}{(D-d)!d!}$$

当原始维度 D 较大时，搜索空间呈指数级增长。例如，当 $D = 100, d = 10$ 时，组合数约为 1.73×10^{13} 。为了在有限时间内找到理想子集，需要采用不同的搜索策略。

1. 穷举搜索 (Exhaustive Search) 计算特征的所有可能组合并逐一评价。虽然能找到最优子集，但总搜索数 $\sum_{d=1}^D C_D^d = 2^D - 1$ 计算量巨大，仅适用于 D 非常小的情况。

2. 单独最优特征组合 对每个特征 x_i 分别进行单独评估，计算其评价得分 $J(x_i)$ ，然后按得分从大到小排序，选取前 d 个特征。这样算法极简，计算量极低。但是完全忽略了特征之间的相关性，无法处理特征冗余。在这个算法中，年龄和出生日期这两个特征都会打高分并被选择，但是这就带来了冗余信息。

3. 序贯搜索算法 (Sequential Search) 本质上这是一种贪心算法，通过每一步的局部最优选择来逼近全局最优。

Algorithm 1 前向序贯选择算法 (SFS)

Require: 特征全集 $X = \{x_1, \dots, x_D\}$, 目标维度 d

Ensure: 最优特征子集 $X_{selected}$

- 1: 初始化 $X_{curr} \leftarrow \emptyset$
 - 2: **for** $k = 1$ **to** d **do**
 - 3: 寻找 $x_j \in X \setminus X_{curr}$, 使得 $J(X_{curr} + \{x_j\})$ 最大
 - 4: $X_{curr} \leftarrow X_{curr} \cup \{x_j\}$
 - 5: **end for**
 - 6: **特点:** 一旦特征被加入，后续步骤无法将其删除。
 - 7: **return** X_{curr}
-

4. 其它序贯变体

- **GSFS (广义 SFS):** 每次不是加入 1 个特征，而是加入 k 个特征，使得组合后性能最优。其计算量比标准 SFS 大。

- **SBS (后向序贯选择)**: 从全集开始, 每次剔除一个特征 $x_j \in X_{curr}$, 使得剩余特征集的评价准则 $J(X_{curr} - \{x_j\})$ 最大。其特点是“一旦删除, 无法增加”。
- **GSBS (广义 SBS)**: 每次从当前集合中剔除 k 个特征。

5. L-R 法 (Plus-L Minus-R) 为了克服 SFS 和 SBS “只进不出”或“只出不进”的单向性缺陷, L-R 法采用交替增减的策略。每次先通过 SFS 增加 L 个特征, 再通过 SBS 减少 R 个特征。若 $L > R$, 最终特征数增加; 若 $L < R$, 最终特征数减少。每次寻优后的集合为 $X_d = X_{d1} + \{x_j\}_L - \{x_i\}_R$ 。广义 L-R (ZL, ZR) 更进一步, 增加 L 个和减少 R 个的过程分 Z 步进行, 通过复杂的步长控制来更深层地搜索特征组合空间。

1.1.2 特征评价准则

定义 1.2 (特征评价准则). 特征评价准则是一个函数 $J(X_{selected})$, 用于定量评估所选特征子集的分类贡献或表达能力。评价准则主要分为两类:

- **可分性度量**: 利用类别间的几何距离、分布重叠度或统计相关性进行打分, 常用于 Filter 框架。
- **学习算法精度度量**: 直接利用分类器的准确率或错误率作为得分, 常用于 Wrapper 框架。

基于距离的可分性判据 这类判据的核心思想是: 好的特征子集应该使得“类内距离尽可能小, 类间距离尽可能大”。这通常依赖于对样本散度矩阵的计算。

1. 基础数据统计量 假设特征选择后的样本向量为 x'_j , 第 i 类的平均向量为 m_i , 总平均向量为 m , 样本总数为 n 。

- **类内散度矩阵 (Within-class Scatter Matrix)**: 表示第 i 类样本内部的离散程度。

$$S_i = \frac{1}{n_i} \sum_{j=1}^{n_i} (x'_j - m_i)(x'_j - m_i)^T$$

- **总类内离散度矩阵 (S_w)**: 各类别散度矩阵的加权和。

$$S_w = \sum_{i=1}^c P_i S_i \quad (P_i \text{ 为第 } i \text{ 类的先验概率})$$

- **类间散度矩阵 (S_b)**: 表示各类别中心相对于总平均值的离散程度。

$$S_b = \sum_{i=1}^c (m_i - m)(m_i - m)^T$$

2. 常用可分性判据 Fisher 鉴别 J_f

- **一维特征 (两类)**: 采用比值形式。

$$J_f = \frac{(m_1 - m_2)^2}{S_1^2 + S_2^2}$$

- **多维特征子集**: 原本也应该是 $J_f = \frac{S_b}{S_w}$, 但是结果是一个矩阵而不是值, 下面是常用的取值:
 - $J_1 = \text{tr}(S_w + S_b)$: 总散度。
 - $J_2 = \text{tr}(S_w^{-1} S_b)$: 类间与类内散度的比率 (最常用)。
 - $J_3 = \ln \frac{|S_b|}{|S_w|}$: 行列式比值的对数。
 - $J_4 = \frac{\text{tr}(S_b)}{\text{tr}(S_w)}$: 迹的比值。
 - $J_5 = \frac{|S_b - S_w|}{|S_w|}$: 差值的比例。

基于概率分布的可分性判据 这个判据依赖于类别在特征空间的概率密度函数。比如水果中西瓜这个特定的类别, 水果有很多特征, 重量、含糖量、外表颜色等, 现在只关注重量这个特征, 西瓜可以有不同的重量, 这就构成了一个概率分布, 橘子也有重量, 也有对应的概率分布, 西瓜一般来说重量自然是比橘子大的多, 所以二者概率密度函数重叠并不高。但是对于梨子和苹果, 二者的重量概率分布函数重叠很高, 我们就很难用重量这个特征区分二者。在实际操作中, 我们很难得到真正的概率分布, 所以一般会通过数据假设概率分布函数形式并通过参数估计决定概率分布函数的具体形式。那么现在一个重要的问题就是如何选择一个合理的概率距离准则 J 。一个合理的概率距离准则 J 应满足:

1. $J \geq 0$ 。
2. 当类别完全不重叠 ($p(x|\omega_1)p(x|\omega_2) = 0$) 时, J 达到最大值 J_{max} 。
3. 当类别完全重合 ($p(x|\omega_1) = p(x|\omega_2)$) 时, $J = 0$ 。

常见的概率距离准则有以下几类:

- **Bhattacharyya 距离**: 衡量概率分布的相似度。

$$J_B = -\ln \int [p(x|\omega_1)p(x|\omega_2)]^{1/2} dx$$

- **Chernoff 距离**: Bhattacharyya 的广义形式。

$$J_C = -\ln \int p^s(x|\omega_1)p^{1-s}(x|\omega_2)dx \quad (0 \leq s \leq 1)$$

- **散度 (Divergence):** 衡量两个分布之间的不对称性差异。

$$J_D = \int [p(x|\omega_1) - p(x|\omega_2)] \ln \frac{p(x|\omega_1)}{p(x|\omega_2)} dx$$

- **Matusita 距离:**

$$J_M = \left[\int (\sqrt{p(x|\omega_1)} - \sqrt{p(x|\omega_2)})^2 dx \right]^{1/2}$$

基于熵的判据（信息论视角） 从信息论的角度看，特征选择的任务可以被视为一个“消灭不确定性”的过程。评价一个特征好坏可以利用“得到这个特征之后，我对判断的困惑程度增加了多少”来理解。这就是**互信息 (Mutual Information, $I(X;Y)$)**。互信息的核心思想是衡量特征 X 与类别 Y 的关联程度，他的一个核心表达式是：

$$h(x, y) = \frac{p(x, y)}{p(x)p(y)}$$

- **独立情况（无价值）:** 若 X 与 Y 完全无关，则联合概率等于概率乘积 ($p(x, y) = p(x)p(y)$)，比值为 1， $\log(1) = 0$ ，互信息为 0。
- **相关情况（有价值）:** 若 X 与 Y 存在关联，比值将偏离 1。可以假设有与判断目标 Y 完全相关的特征 X ，则离散情况下，分布式为对角矩阵。这个时候 $p(x, y) = p(x) = p(y)$ ，对角元素为 $1/p(x)$ ，说明特征 x 可以很好的决定 $y = x$ 时的信息；对于非对角元素，结果是 0，说明完全不提供 $y \neq x$ 处的信息。

对于特征 X 和类别 Y ，互信息的计算取决于数据的类型：

(1) 离散情况 (Discrete Case) 当特征和类别均为离散取值时，求和是对特征 X 的所有可能状态（或区间）以及所有类别 Y 的组合进行遍历：

$$I(X;Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

注：在实际操作中，概率 $p(x, y)$ 通常通过统计样本落在特定组合格子的频率来估算。

(2) 连续情况 (Continuous Case) 若特征 X 是连续数值（如温度、电压），公式转化为在特征空间上的二重积分：

$$I(X;Y) = \int_{\mathcal{X}} \int_{\mathcal{Y}} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} dx dy$$

注：由于连续概率密度函数 $p(x, y)$ 较难直接获得，工程上常通过“分箱 (Binning)”将连续特征离散化，再利用离散公式进行计算。

另外一个重要的判据是条件互信息 (Conditional Mutual Information) 在已知特征 Z 的前提下, 特征 X 为判断类别 Y 提供的额外信息量:

$$I(X; Y | Z) = \sum_{z \in \mathcal{Z}} p(z) \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y | z) \log \frac{p(x, y | z)}{p(x | z)p(y | z)}$$

这常用于检测特征冗余: 如果已知 Z 后, X 提供的额外信息几乎为 0, 则说明 X 相对于 Z 是冗余特征。

1.1.3 特征选择方法

Filter 方法 (过滤式) Filter 方法不依赖于后续的学习算法 (如分类器)。它直接根据数据的统计特性 (如距离、相关性、互信息等) 构建评估函数, 对特征子集进行评价。其逻辑是“先过滤, 后训练”, 即先从原始特征中选出最优子集, 再将其输入学习器。其计算复杂度低, 执行效率高, 适用于大规模数据集。但是选择过程与学习算法解耦, 可能导致选出的特征子集在特定模型上性能不是最优, 且容易忽略特征间的冗余信息。

例 1.3 (基于距离评价的特征选择流程). 假设有两类 3 维数据 (W_1 红色, W_2 绿色), 每个类别包含 3 个样本。我们的目标是从 3 个属性 x_1, x_2, x_3 中选择最有用的特征。

1. **1 维特征评估:** 对每个特征独立计算 Fisher 判据 $J_f = \frac{(m_1 - m_2)^2}{S_1^2 + S_2^2}$ 。通过观察数据分布, 发现 x_2 在两类间的均值差异最大且类内分布紧凑。计算结果排序为: $J(x_2) > J(x_3) > J(x_1)$ 。
2. **子集组合评估:** 当需要选择 2 维特征子集时, 计算所有可能组合 (如 $\{x_1, x_2\}, \{x_2, x_3\}$ 等) 的矩阵迹判据 $J_2 = \text{tr}(S_w^{-1} S_b)$ 。
3. **结论:** 选择 J_2 得分最高的子集 (如示例中的 53.2917 分组合), 该子集在空间中能产生最佳的类别分离度。

例 1.4 (Relief 算法). Relief 算法通过计算特征对于“同类近邻”的凝聚能力和“异类近邻”的区分能力来评估特征权重。

Algorithm 2 Relief 算法流程

Require: 样本集 D , 迭代次数 m , 阈值 t **Ensure:** 特征权重向量 w

```
1: 初始化所有特征权重  $w_i \leftarrow 0$ 
2: for  $i = 1$  to  $m$  do
3:   随机抽取一个样本  $x$ 
4:   在  $x$  的同类中寻找最近邻样本  $z^+$  (Near-hit)
5:   在  $x$  的异类中寻找最近邻样本  $z^-$  (Near-miss)
6:   for 每个特征  $j$  do
7:      $w_j \leftarrow w_j - \text{diff}(x_j, z_j^+)^2 + \text{diff}(x_j, z_j^-)^2$ 
8:   end for
9: end for
10: return 满足  $w_j > t$  的特征集合
```

这个算法也是 Filter 的，因为 Filter 的本质是直接选取某种方式，独立的评价某个特征的能力，并打分排序选择。再看 *Relief* 算法，这个算法对每个特征的打分结果就是最终的权重，以某个特征是否可以合理的“推开非同类点”而“拉回同类点”进行打分。

对 *Filter* 算法而言，他并不依赖于具体的模型，不需要在模型上验证，所以是一种相对而言十分高效的算法，一般会先用 *Filter* 进行粗筛，其他算法进行细筛。

Wrapper 方法 (包裹式) Wrapper 方法（包裹式方案）的核心思想是将后续的学习算法（如分类器）直接作为特征评价准则的一部分。在该方案中，学习算法被视为一个“黑盒”，特征选择算法不断地产生不同的特征子集，并将其输入该黑盒中进行训练，最终利用模型在验证集上的性能（如准确率或错误率）作为该子集的评价得分。

- **优点:** 针对特定模型“量身定制”，能够捕捉到特征与学习算法之间的相互作用，因此选出的特征子集通常具有更好的预测性能。
- **缺点:** 计算复杂度极高。由于每评估一个子集都需要完整地训练一次模型，在处理高维数据或训练耗时较长的模型时（如深度学习），其执行速度非常缓慢。

例 1.5 (LVW (Las Vegas Wrapper) 算法). LVW 是一种基于拉斯维加斯随机化策略 (Las Vegas Algorithm) 的包裹式特征选择方法。其特点是在搜索空间内随机采样特征子集，并通过模型精度来更新当前最优子集。其更

新逻辑不仅关注错误率的降低，还兼顾了特征数量的精简（奥卡姆剃刀原则）。

Algorithm 3 LVW (Las Vegas Wrapper) 算法流程

Require: 数据集 D ，学习算法 $\mathcal{L}$ ，停止准则（控制参数 T ）

Ensure: 特征子集 A^*

```

1: 初始化：当前最优错误率  $E \leftarrow \infty$ ，最优子集  $A^* \leftarrow A$ （全集），当前
   维度  $d \leftarrow |A|$ ，计数器  $t \leftarrow 0$ 
2: while  $t < T$  do
3:   随机产生特征子集  $A'$ 
4:   计算子集维度  $d' \leftarrow |A'|$ 
5:   通过交叉验证（Cross-Validation）评估学习算法  $\mathcal{L}$  在  $A'$  上的错误
     率  $E'$ 
6:   if  $(E' < E)$  或者  $(E' = E \text{ 且 } d' < d)$  then  $\triangleright$  性能提升或维度更精
     简
7:     更新  $t \leftarrow 0$ 
8:     更新  $E \leftarrow E'$ ， $d \leftarrow d'$ ， $A^* \leftarrow A'$ 
9:   else
10:     $t \leftarrow t + 1$   $\triangleright$  连续  $T$  次尝试无提升则停止
11:   end if
12: end while
13: return 最优特征子集  $A^*$ 

```

细节说明：

- **停止准则：**算法在连续 T 次尝试中未能找到更好的子集时，即认为达到了局部最优或搜索已饱和，从而停止。
- **双重优化目标：**该算法第 9 行的判断逻辑体现了包裹式方案的精髓——首要目标是最小化泛化错误率 E' ，次要目标是在性能持平时尽可能减少特征数量 d' ，以提升模型的可理解性和降低过拟合风险。

嵌入式方法 嵌入式方法（Embedded Methods）将特征选择过程与学习器的训练过程融为一体，在模型的目标函数中直接引入正则化惩罚项。通过优化带有约束的目标函数，模型在学习过程中会自动调整特征权重，使不重要特征的权重收缩至零，从而实现特征的自动选择。

- **优点：**计算效率高（只需一次训练过程），且由于选择过程直接受模型性能驱动，其准确率通常优于 Filter 方法。
- **缺点：**高度依赖特定的学习器模型（如线性模型），通用性相对较弱。

例 1.6 (LASSO 和岭回归). 一般的损失函数就是误差项, 而和是的正则化引入惩罚项目, 不仅可以避免过拟合还能实现特征的“稀疏化”去除冗余特征。一般目标函数表示为:

$$\min_w \underbrace{\sum_{i=1}^m (y_i - w^T x_i)^2}_{\text{平方误差项 (RSS)}} + \lambda \underbrace{\|w\|_n}_{\text{L1 惩罚项}}$$

其中 λ 是调节因子, 控制模型在“预测精度”与“特征稀疏性”之间的平衡。

加入惩罚项之后, 对误差项影响不大的权重会因为乘法项的存在而减小, 事实上, 如果存在相关性很高的特征, 也可以通过这个算法消除冗余。一个需要关注的问题是, 不同的正则项选择会影响算法是否能进行特征的真正系数使得权重 $w = 0$ 。取 w_1, w_2 二维权重空间, 我们把损失函数分为两个部分, 其一是误差项, 因为特征相关性图中其等高线是椭圆, 其二是正则项, 不同正则项的等高线不同。对于某一个特定的误差等高线, 它更容易在权重非零的时候与更小的 L_2 等高线达到平衡, 但对于 L_1 等高线, 权重更倾向于在坐标轴上达到平衡。

由于 L_1 范数在原点处不可导 (尖角特性), 无法直接使用标准的梯度下降法。通常采用 近端梯度下降 (Proximal Gradient Descent, PGD) 等优化算法, 通过软阈值化操作来处理非导数点, 确保权重的稀疏性更新。

1.2 特征提取

1.2.1 特征提取概述

特征提取就是“取其精华, 去其糟粕”, 寻找能容纳数据的最小的那个线性空间。

特征提取是从原始高维空间到低维空间的映射 $R^n \rightarrow R^m$, 与特征选择不同的是, 在这个过程中原始的特征被线性组合形成了新的特征。有两个看待特征提取的视角, 可以通过“观众对电影打分”的例子来直观理解: 如果我们研究观众对电影的喜好来拍电影, 我们当然可以直接用观众对所有电影的评分矩阵, 而后拍一部和评分高的哪些矩阵相近的。但是电影的数量太多太多了, 而且很多电影具有相同的元素并且有可能会误把没有意义的特征纳入考量, 这就是说, 以具体的电影作为基向量会造成很大程度的信息冗余, 带来不必要的计算麻烦, 我们应该提取那些特征。

投影视角 ($L = XW$) 从左往右看, 我们希望从已知的打分情况 X 中, 提炼出用户对电影各个元素的隐性偏好 L 。这可以理解为: 将“人”的原始特征 (打分向量) 往“电影元素”的特征向量上进行投影, 看两者的相近

$X = LR$ 成立当
 $r(L)=r(R)=r(X)$

程度。横向量乘以一个矩阵，结果可以描述为这个横向量的恒特征在矩阵每个列特征上的投影（相似程度）是多少，那么 W 的恒特征是“电影元素”列特征是“具体电影名”。

矩阵分解视角 ($X \approx LR$) 设 X 是 m 个观众对 n 部电影的打分矩阵，我们可以把评分拆解为两个部分，第一个是观众对于电影不同特征的喜好程度（科幻、悬疑、动作而而）矩阵 L ，第二个是电影所含有的不同特征的程度矩阵 R 。这里分解的核心是：“用户对每部电影的喜好程度决定于电影有多大程度上占据了用户喜好的电影特征。”

两种表示等价当 W
 和 R 互为广义逆矩阵，易证

我们想要的核心结果就是 L ——观众对不同电影特征的喜好程度，有了这个矩阵，了解观众的喜爱就容易多了。

$X = LR$ 总成立吗？其实是在问是不是任意低维的数据都能完全不失真的还原原始数据？答案是否定的，如果维度低于容纳数据的最小子空间的维数，就一定会损失信息。等号成立的条件是三个矩阵秩相等， $r(L)$ 表示原始数据的真正自由度， $r(R)$ 表示其行空间的向量维数，这二者相乘保证了以 R 的行向量展开 L 的信息不会损失表示数据的自由度，最后等于 $r(X)$ ，说明不需要丢弃自由度表示信息

关于逆矩阵这个地方，
 其实我没有彻底想明白，留一个 *TODO*

我们一般要求 R 是正交的，虽然线性无关的向量组就足以构成基，但是正交基有更大的好处，如果正交矩阵原本表示横特征 A 在特征 B 上的分布情况，那正交矩阵的转置就直接是列特征 B 在横特征 A 上的分布，而对于一般的非正交矩阵，因为基向量之间存在耦合其转置还需要乘以一个解耦的矩阵（结果就是矩阵的逆）才能真正表示特征 B 在 A 上的分布。比起求逆，转置大幅降低了运算量。另一个有意思的现象是 $W = R^T$ ，虽然这个结果可以通过简单的矩阵计算得到，但是结合我们之前对矩阵含义的直观诠释，也可以赋予这个等式一个直观的理解。从投影视角来看，矩阵 W 是横特征“具体电影名”在列特征“电影元素”上的分布，从矩阵分解视角看， R 应当表示“电影特征”在“具体电影名上的分布”，正交矩阵的转置刚好能表示交换特征后的分布，所以二者相等。

1.2.2 主成分分析 (PCA)

PCA 只是想优化均方误差来构建最优子空间：

$$\min_{L,R} \|X - LR\|^2 \quad (1.2.1)$$

但是在两种观点等价时，可以得到这个优化过程最后被归为寻找散度矩阵的特征向量。

定理 1.7. 主成分分析的目标函数具有等效性，即最小化重构误差等价于最大化投影方差：

$$\min_{L,R} \|X - LR\|^2 \iff \max_w w^T S w$$

注意，我们最终求散度矩阵需要先把数据进行标准化（减去标准差并除以均值），否则特征向量会有一个从原点指向质心的部分。

证明.

$$\begin{aligned} \min_{L,R} \|X - LR\|^2 &= \min_w \sum_i |x_i^T - l_i w^T|^2 \\ &= \min_w \sum_i d_i^2 \\ &= \min_w \sum_i (|x_i|^2 - |l_i w^T|^2) \\ &= \max_w \sum_i |l_i w^T|^2 \\ &= \max_w \sum_i l_i^2 |w|^2 \\ &= \max_w \sum_i l_i^2 \\ &= \max_w \sum_i (x_i^T w)^2 \\ &= \max_w \sum_i w^T x_i x_i^T w \\ &= \max_w w^T \left(\sum_i x_i x_i^T \right) w \\ &= \max_w w^T S w \end{aligned}$$

□

散度矩阵（协方差矩阵）描述了数据的离散程度。其特征向量方向即为数据分布最散、特征最明显的投影方向。通过保留大特征值对应的维度，PCA 实际上在挖掘数据自身的固有性质，并滤除了随机噪声，从而保证了结果的泛化能力。

Algorithm 4 主成分分析 (PCA) 算法流程

Require: 样本集 X (每行一个样本), 目标维度 d

- 1: **中心化 (标准化):** $x_i \leftarrow x_i - \bar{x}$ 。若不中心化, 提取的特征向量会叠加一个原点到数据质心的偏移向量。
 - 2: **求协方差矩阵:** 计算 $S = X^T X$ 。
 - 3: **特征值分解:** 计算 S 的特征值 λ_i 并按从大到小排序。
 - 4: **构造变换矩阵:** 取前 d 个非零特征值对应的特征向量组成 W 。
 - 5: **降维表示:** 计算训练样本的投影结果 $L = XW$ 。
-

1.2.3 SVD 分解

SVD 分解最主要的特征时可以通过奇异值近似原始矩阵, 对于任意一个 $M \times N$ 矩阵 A , $A^T A$ 是半正定实对称矩阵可以对角化, 且特征值大于 0。设 Σ 是对角矩阵开根号, V 是对角化矩阵, 则有:

$$V^T A^T A V = \Sigma^2 \quad (1.2.2)$$

参加知乎文章

1.2.4 LLE (Locally Linear Embedding)

一种看待高维数据特征稀疏的观点是引入**流形 (Manifold)**: 即高维观测数据实际上分布在嵌入于高维空间的低维流形上。利用流形的**局域欧几里得特性 (Locally Euclidean)** 可以辅助特征提取——由于流形在局部具有平坦的性质, 每个数据点都可以由其近邻点线性表示。因此, 我们可以先计算出每个点相对于其近邻的重构权重。

随后是流形的“铺平”过程: 正如将一张卷曲的纸张展开, 在此过程中局域欧几里得性质保持不变, 其线性重构权重也应保持不变。基于此, 我们可以寻找一组新的低维坐标来重建流形, 从而在保持局部几何结构的前提下实现降维。

Algorithm 5 局部线性嵌入 (LLE) 算法流程

- 1: **输入:** 高维数据集 $\{x_1, x_2, \dots, x_n\}$, 目标维数 d , 近邻数 k 。
- 2: **Step 1:** 寻找近邻
- 3: 对于每个样本点 x_i , 通过欧氏距离确定其 k 个最近邻点的集合 N_i 。
- 4: **Step 2:** 计算局部重构权重 W
- 5: 最小化重构误差函数:

$$\min_w E(W) = \sum_{i=1}^n \left\| x_i - \sum_{j \in N_i} w_{ij} x_j \right\|^2$$

约束条件: $\sum_{j \in N_i} w_{ij} = 1$, 且若 $x_j \notin N_i$, 则 $w_{ij} = 0$ 。

- 6: **Step 3:** 计算低维嵌入坐标 Y
- 7: 固定权重 w_{ij} , 在低维空间寻找坐标 $\{y_1, y_2, \dots, y_n\}$, 最小化损失函数:

$$\min_Y \Phi(Y) = \sum_{i=1}^n \left\| y_i - \sum_{j \in N_i} w_{ij} y_j \right\|^2$$

约束条件: $\sum y_i = 0$ 且 $\frac{1}{n} Y Y^T = I$ 。

- 8: **输出:** 低维表示 $Y = \{y_1, y_2, \dots, y_n\} \in \mathbb{R}^{d \times n}$ 。

定理 1.8. 最小化 LLE 的重构损失函数等价于求解矩阵 $M = (I - W)^T(I - W)$ 的特征值分解问题。在满足去中心化和单位协方差约束下, 最优低维映射由矩阵 M 对应的第 2 到第 $d+1$ 小特征值的特征向量构成。

证明. 首先将损失函数 $\Phi(Y)$ 改写为矩阵迹 (Trace) 的形式。令

$$Y = [y_1, y_2, \dots, y_n] \in \mathbb{R}^{d \times n}$$

, 则重构项可以表示为矩阵 Frobenius 范数的平方:

$$\Phi(Y) = \|Y - YW\|_F^2 \quad (1.2.3)$$

利用性质 $\|A\|_F^2 = \text{tr}(AA^T)$, 我们可以展开上式:

$$\Phi(Y) = \text{tr}((Y - YW)(Y - YW)^T) = \text{tr}(Y(I - W)(I - W)^T Y^T) \quad (1.2.4)$$

令算子矩阵 $M = (I - W)^T(I - W)$, 由于 M 是对称半正定矩阵, 优化问题简化为:

$$\min_Y \text{tr}(YMY^T) \quad \text{s.t.} \quad \frac{1}{n} Y Y^T = I \quad (1.2.5)$$

为了求解该受约束的极值问题, 引入对角乘子矩阵 $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$, 构建拉格朗日函数:

$$L(Y, \Lambda) = \text{tr}(YMY^T) - \text{tr}\left(\Lambda \left(\frac{1}{n} Y Y^T - I\right)\right) \quad (1.2.6)$$

对 Y 求矩阵梯度并令其为零：

$$\frac{\partial L}{\partial Y} = 2YM - \frac{2}{n}\Lambda Y = \mathbf{0} \implies MY^T = Y^T \left(\frac{1}{n}\Lambda \right) \quad (1.2.7)$$

这表明 Y^T 的列向量必须是 M 的特征向量。此时损失函数的值为：

$$\text{tr}(YMY^T) = \text{tr}\left(Y(Y^T \cdot \frac{1}{n}\Lambda)\right) = \text{tr}\left(\frac{1}{n}YY^T\Lambda\right) = \text{tr}(I \cdot \Lambda) = \sum_{i=1}^d \lambda_i \quad (1.2.8)$$

根据广义瑞利商 (Generalized Rayleigh Quotient) 理论，要使迹最小，应选取矩阵 M 最小的 d 个特征值对应的特征向量。

最后考虑矩阵 M 的零特征值问题。由于 W 每一行元素之和为 1，满足 $W\mathbf{1} = \mathbf{1}$ ，则：

$$(I - W)\mathbf{1} = \mathbf{0} \implies M\mathbf{1} = (I - W)^T(I - W)\mathbf{1} = \mathbf{0} \quad (1.2.9)$$

这意味着 0 始终是 M 的一个特征值，且其特征向量为全 1 向量 $\mathbf{1}$ 。该解代表所有数据点坍塌到低维空间同一点。由于约束条件 $\sum y_i = 0$ 要求 $Y\mathbf{1} = \mathbf{0}$ ，即 Y 必须与该平凡特征向量正交，因此我们必须舍弃特征值 0，取第 2 小到第 $d+1$ 小特征值对应的特征向量作为输出。□

1.2.5 NMF (Non-negative Matrix Factorization)

算法概述 非负矩阵分解 (NMF) 是一种在特征提取和降维领域极具特色的矩阵分解方法。其核心思想在于：对于一个非负的原始数据矩阵 V ，我们寻找两个同样具有非负约束的低维矩阵 W (基矩阵) 和 H (系数矩阵)，使得 $V \approx WH$ 。与 PCA 等允许存在负值系数的线性方法不同，NMF 强迫模型通过“只加不减”的逻辑来重构数据。这种非负性约束在物理和心理学意义上对应了“部分构成整体” (Parts-based representation) 的认知直观：在重构过程中，特征之间只能进行正向的叠加而不能相互抵消。这使得 NMF 提取出的基向量往往具有极强的可解释性，例如在人脸识别中，它能自动发现眼睛、鼻子等局部零件特征；在文本处理中，它能提取出代表特定主题的词簇。

目标函数与启示 由于 $V \approx WH$ 是一种近似关系，我们需要定义目标函数来衡量重构误差。根据对数据分布和噪声特性的不同假设，通常考虑以下两种损失函数：

1. **欧氏距离 (Frobenius 范数误差)**：目标是最小化 $\|V - WH\|_F^2 = \sum_{i,j} (V_{ij} - [WH]_{ij})^2$ 。这种准则在数学上等价于假设观测数据受到高

斯噪声的影响，它追求重构结果在数值绝对偏差上的最小化，适用于大多数通用特征提取场景。

2. **K-L 散度 (I-divergence)**: 目标是最小化 $\sum_{i,j} (V_{ij} \ln \frac{V_{ij}}{[WH]_{ij}} - V_{ij} + [WH]_{ij})$ 。这种准则在统计学上通常与泊松分布相关联，对于处理计数型数据、概率分布数据（如文档中的词频分布）具有更好的建模效果，能更好地捕捉数据的结构化信息。

这两种目标函数通常配合“乘性更新规则”（Multiplicative Update Rules）进行迭代求解。这种规则的精妙之处在于，只要初始矩阵 W 和 H 是非负的，更新过程中的乘法运算会自动保证非负约束始终成立，无需复杂的约束优化算子，在保证数值稳定性的同时具有良好的收敛特性。

几何直观 从几何角度来看，NMF 的非负约束将高维数据映射到了一个由非负基向量张成的凸锥（Convex Cone）中。不同于线性空间中点可以分布在坐标轴的任意象限，NMF 将数据点限制在第一卦限的一个特定区域内。由于基向量和组合系数均不能为负，观测点必须落在这些基向量构成的“锥角”内部。这种受限的几何结构赋予了 NMF 极强的聚类倾向：数据点往往会向某一个或几个特定的基向量靠拢，使得每个基向量自然而然地成为了一个潜在类别的中心。因此，NMF 得到的低维表示不仅是数据的压缩，更是一种天然的聚类结果，能够清晰地揭示高维数据中隐藏的语义结构。

1.2.6 KPCA

KPCA

I. 形式

一维 $\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$ PCA 无解 (线性不可分)

二维 $\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$ PCA 可解

PCA 从考察 (设为特征化)

$\frac{1}{N} \sum x_i x_i^T$ 变为 $\frac{1}{N} \sum \phi(x_i) \phi(x_i)^T = C$

II. 求 C 特征值

设 $Cv = \lambda v$ 有

$(\frac{1}{N} \sum \phi(x_i) \phi(x_i)^T) v = \lambda v$

数

故 C 特征向量 v 可由 $\phi(x_i)$ 展开, 设

$v = \sum \alpha_i \phi(x_i)$ 有

$\frac{1}{N} \sum \phi(x_i) \phi(x_i)^T (\sum \alpha_j \phi(x_j)) = \lambda \sum \alpha_j \phi(x_j)$

左乘 $\phi(x_k)^T$ 有

$\frac{1}{N} \sum \phi(x_k)^T \phi(x_i) (\sum \alpha_j \phi(x_j)^T \phi(x_j))$

$= \frac{1}{N} \sum (\phi(x_k)^T \phi(x_i)) (\sum \alpha_j \phi(x_j)^T \phi(x_j))$

定义 $K_{ij} = \phi(x_i)^T \phi(x_j)$

$\downarrow$ K 矩阵 $N \times N$ 维, 由数据决定

化为

LHS: $\frac{1}{N} \sum K_{ki} \sum \alpha_j K_{ij}$

$\Rightarrow K \cdot K \alpha / N$

RHS: $\frac{1}{N} \sum K_{kj} \alpha_j \Rightarrow \frac{1}{N} K \alpha$

设 K 可逆, 由

$\frac{1}{N} K \alpha = \frac{1}{N} K \alpha$ 得

$K \alpha = N \lambda \alpha$

即 C 的特征值求解化为 K 特征值

求解: $K \alpha = \lambda \alpha$ (N 无所谓)

得到 λ 后, α 可由 C 求得

III. KPCA 流程

1. 取核函数 $K(x, y) = \exp(-\|x - y\|_2^2 / 2\sigma^2)$ 得 K 矩阵
2. 将 K 特征化, 放到中心

$\bar{K} = K - \frac{1}{N} K \mathbf{1} \mathbf{1}^T - \frac{1}{N} \mathbf{1} \mathbf{1}^T K + \frac{1}{N} \mathbf{1} \mathbf{1}^T K \mathbf{1} \mathbf{1}^T$

$\mathbf{1} = \begin{pmatrix} 1 & \dots & 1 \\ 1 & \dots & 1 \\ \vdots & & \vdots \end{pmatrix}$

3. 特征值求解, 得 λ_i, α_i
4. λ_i 降序, 并取前 d 个 (留下 d 个最大的特征)
5. 得到特征方向 (α_i 的方向)

即 $y = \sum \alpha_i \phi(x_i)$ (注意, 这是 K 集特征值)

$\Rightarrow U = [u_1, u_2, \dots, u_d]$ 下的特征向量, $\alpha_1, \alpha_2, \dots, \alpha_d$ 对应同一特征值

数据在该方向上

示为 $y = U^T \phi(x) = [\sum \alpha_i \phi(x_i) \phi(x_k), \dots]$

$\downarrow$ K_{ik}